No
Date

1) Word Embeddings - Theory & Analysis.

Word Embedding :-

In simple words, word embedding is the process of turning words into numbers.

Computers cannot read raw ~~text~~, it can represent it but it cannot understand it.

Word embedding is process of applying algorithms to turn ~~word~~ into numerical vectors, These vectors carry the meaning of the word so that computer can understand it.

One-hot encoding :-

To convert textual data to numerical forms, in one-hot encoding, each unique world is mapped into a binary vector where a single position has value 1 & others are zero.

The length of a vector is vocab size so every word can be assigned a unique vector.

for example corpus : = I like bread.

$$I = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad like = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \quad bread = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

No
Date

Dense- word embedding :-

Dense-word embedding converts the words of a text into continious vectors of real numbers where most elements are non-zero values. In dense word-embedding usually vector-length / dimension is less than vocab size. Each value of a vector contributes to the word's meaning.

One-hot Encoding	vs	Dense vector encoding.
The vocab size is very high and as one-hot encoding creates vectors with high dimensionality.		Dense vector encoding uses predefined, fixed vector size which is usually very low than vocab size.
Only a single dimension contributes to the whole word meaning.		Most of the dimensions contribute to a vector's meaning.
It does not capture any semantic meaning of word.		It can capture the semantics of word.
No representation for unseen words.		Unseen words can be handled using some methods.
The encoding is manually defined nothing is learned.		Encoding is learned based on corpus.

NISSAN

No
Date

Compare Word2Vec (CBOW vs SKIP-Gram), GloVe and ~~FAST~~ FastText.

CBOW (Continious Bag of words)

- CBOW is a supervised learning method to generate word embedding
- In CBOW we define a fixed context window around a word.
- For example in "I like bread very much." with a context window of 2,

(I like | bread | very much).

⇒ In CBOW, the ~~main~~ learning task is to predict the middle word based on the surrounding context.

Input: "I" "like" "very" "much"
Label: "bread"

⇒ For inputs, to the machine learning models for training, we use simple one-hot encoding to encode them.

⇒ During training with, $d_1 \ldots d_n$ documents,

- Generate a dictionary of all words,
- For each document & each word in document create training example with context as input and the word as target label.
- One-hot encode all indices.

NISSAN

No
Date

⇒ After training, the learned weights are used to create embedding vectors.

~~The learned output~~

Obtaining a word's embedding:

When training the neural network a three layer network is created.

The first layer (input) cobines the context words by adding the one hot encodings into a single vector, this makes the input size V (vocab count)

~~The hidden layer is choosen by from a range~~

The hidden layer has an appropriate number of neurons, selected based on complexity of the corpous. D → (Hidden layer size). This hidden layer size gives the dimensionality of the embedding.

After training, each row of the learned weight (VXD) matrix gives the embedding for the corresponding word,

if V_w is the one hot-encoded word,

Dense embedding $E_w = V_w * W$

where W is the weight

No
Date

Skip-Gram method:-

Skip-Gram is another method for generating vectors from words. It is similar to CBOW in the sense that, they both treat generating the embeddings as a learning problem and the machine learning is done on a fixed context window of text, the difference in the two is in what the neural network is trying to learn.

In CBOW, the NN learns the middle word based on the context but it is opposite in skip gram, where the middle word is used to predict the context words.

Training step!-

For each context input is

Input = middle word.

Output = surrounding words in context.

I am very tired today;

C = 2

then, Input = "very"

Output = ["I", "am", "tired" "today"];

The NN is trained for all the words and its surrounding context in the corpous and we get a weight matrix after training.

During training, each pair of words in the context is used to generate a training example.

NISSAN

No
Date

Getting the embeddings :-

→ As in CBOW, each row in the weight matrix VXD corresponds to the embedding of the word.

CBOW	vs	Skip-Gram
→ CBOW is best for large data and frequent word		Skip-gram is better for small data & infrequent word.
→ It is faster to train as a single training step for each word in context.		→ Takes more time to train as each word pairs have separate training step for each word.

GloVe :-

GloVe (Global vectors for word embedding) is a count based method for generating word embeddings.

In GloVe, word-word co-occurence matrix X is first built over the whole corpus the algorithm then tries to learn the embeddings that closely approximates the log of the counts.

Glove uses the global statistics rather than local context.

NISSAN

No
Date

Fast Text:

Fast-text is an extention to the ~~N-grams~~ Word2Vec skip-gram model. Rather than representing the word as a vector, it divides the word into a character n-gram $n \in [3, 6]$ usually.

Only the middle word is converted to this ~~n-gram~~ character n-gram, the vector representation of the whole word is the sum of the n-grams.

As, the input to the network/model is now divided into character n-grams, this method can understand the semantics of apple vs apples, go vs going etc.

	Word2Vec	GloVe	FastText
Method	Local predictive	Global predictive	Local predictive
Granularity	Full word	Full word	Character n-gram.
Unseen words	Fails	Fails	Handles
Semantics	Runs & Run is completely different	Run & Runs is completely different	can ~~treat~~ understand words & its suffixes/prefixes

Other than this all of the methods are static non-contextual word embedding algorithms.

NISSAN

No
Date

Semantic similarity :-

Embedding vectors, capture the semantics of words in many dimensions,

In some high dimensional space, if any two vectors are pointing in the same direction it usually means they are synonymous or semantically accurate.

To measure the similarity, cosine similarity is usually used as a metric,

the formula is,

$$\cos(u,v) = \frac{u \cdot v}{|u|\,|v|}$$

if the value of ~~two~~ vectors are close two one, it means they ~~are~~ represent synonymous / related words.

- cos ("king", "queen") → high
- cos ("king", "car") → low

The embeding space can also allow for vector operations,

diff = vector("king") - vector("queen")

vector("man") + diff = ~~queen~~ vector("woman");

We observe, such things in the vector space, ~~that~~ where subtracting king & queen resulted in a vector representing gender difference, we can add that to man to get a vector close to women.

NISSAN

No
Date

Dimensionality in embedding representation;

Dimensionality is a hyper parameter in ~~vector~~ word embedding generation, if we choose a low dimension, the vectors generated won't be able to capture semantic meanings of the task well. With increase in dimensionality, the ability to encode more information / semantics increases but at some point a higher dimension will just add noise and overfit the training corpus.

The dimensionality also determines the cost of embedding, smaller dimensions require a less cost while higher dimensions ~~need~~ consume more resources.

As, dimensionality depends on how complex the semantics that are needed to be learned are and also on the cost, determining it is very task dependent.